

캡스톤디자인과창업프로젝트B

탐조 입문자를 위한
Image Classification 기반 실시간 새 종 판별
및 새 관측 기록 제공 서비스
- Birder -

지도교수 이형준

TEAM10 JL (산학)

2076422 최이준

2276330 최유선

2317024 이윤성

< 목 차 >

I . Team 10 JL Info	1
II . Birder-Summary	
1. Birder의 문제 정의	2
2. 기존 서비스와의 비교	3
3. Birder에서 제공하고자 하는 서비스	4
4. 기대 효과 및 의의	5
5. 주요 기능 리스트	6
III . Project-Design	
1. 요구사항정의서	8
2. Birder의 시스템 구성도	10
3. 주요 엔진 및 기능 설계	14
4. 주요 기능의 구현	16
5. 기타	17

I . Team 10 JL Info

1. 프로젝트명

탐조 입문자를 위한 Image Classification 기반 실시간 새 종 판별 및 새 관측 기록 제공 서비스

2. 팀 정보 및 팀 구성원

팀명: JL

- 최이준(2076422): 팀리더, BE-DB 담당, 인증 시스템 및 사용자 관리 API 백엔드 구현, Supabase 기반 서버·DB 아키텍처 설계 및 연동, AI 판별 이미지 업로드 및 결과 데이터 처리, DB response 성능 검증
- 최유선(2276330): BE-AI 담당, 로그인/회원가입/회원탈퇴 인증 백엔드 개발, 종·지역 기반 검색 API 설계 및 구현, OpenAI 기반 AI 검색 로직 구현, AI 결과 정확도 검증
- 이윤성(2317024): FE 총괄 담당, 사용자 인증 및 계정 관리 프론트엔드 구현, 종·별·지역별 검색 UI 및 결과 페이지 개발, AI 검색 결과 화면 및 API 연동, FE-BE request response 사이 연결 검증

II. Birder-Summary

1. Birder의 문제 정의

탐조란 새를 관찰하는 활동으로, 탐조를 하는 사람을 탐조인이라고 한다. 탐조인은 주로 새의 행동, 깃털, 색상 등 특징을 관찰하며, 신체적 특징 뿐 아니라 섭식 활동, 서식지 등 행동을 관찰하여 새의 종류를 판별하고 관측 기록을 남긴다. 탐조에는 새 종 판별(이하 동정同定)이 필히 동반되는데, 이때 보통 탐조인은 눈으로 관측한 새 종의 외형과 일치하는 새 종을 새 도감을 이용해 직접 동정해야한다. 이러한 탐조의 특이점은 탐조 입문자로 하여금 다양한 어려움을 겪게 한다. 탐조 입문자가 겪는 대표적인 어려움은 다음과 같다.

첫째, 즉각적인 동정이 불가하다. 새 도감은 ‘과(科)’별로 구분되어 있어 탐조 시 과별 특징을 포착하는 능력이 요구된다. 하지만 탐조 입문자는 새의 어떤 특징을 기준으로 잡아야 할지도 모를뿐더러, 어떤 특징이 과를 분류하는지도 알지 못하기 때문에 탐조 시 발견한 새에 대한 동정이 불가능하다.

둘째, 탐조 기록을 관리하는데 불편함이 크다. 현재는 탐조 후 관측 위치, 시간, 종을 기준으로 사진 등 탐조 파일을 관리하는데, 이는 탐조인이 직접 분류하고 저장해야 하는 것으로 정확도도 떨어지고 기준점도 탐조인마다 달라 효율성이 떨어진다.

셋째, 최신 관측 정보의 실시간 공유가 어렵다. 살아있는 생물인 새를 관측하는 탐조 특성과 더불어 계절마다 이동하는 철새도 있는 새 종 특성상 탐조에는 최신 정보가 중요하게 작용한다. 하지만 탐조인들의 전용 커뮤니티가 부실하다는 점, 커뮤니티에서도 최신 정보를 실시간으로 확인하기 어렵다는 점은 현재 탐조에 있어 제한점이다.

II. Birder-Summary

2. 기존 서비스와의 비교

탐조가 주류 취미활동이 아닌 만큼 탐조 관련 서비스는 다양하거나 그 양이 많지 않은 편이다. 시중에 제공되고 있는 탐조인이 사용할 만한 서비스는 'eBird Mobile'과 'Merlin Bird ID' 그리고 작년 11월에 출시된 '새록'이다.

첫째로 'eBird Mobile'은 Cornell Lab에서 제공하는 eBird 서비스를 모바일 앱으로 제공하는 어플로, 전 세계를 범위로 서비스를 제공하며 종별 출현하는 위치 검색과 위치별 출현하는 종 검색이 가능하다는 점이 특징이다. 현재 서비스 중 가장 방대한 양의 데이터를 가지고 있는 eBird 서비스를 기반으로 제공하기 때문에 데이터의 질이 타 서비스에 비해 높은 강점이 있다. 하지만 이 서비스는 종 판별 기능, 사용자의 탐조 기록 관리 기준에서 부실해 DB 제공으로써의 강점은 있지만 탐조인의 전체적인 탐조 장벽을 낮추는 점에서는 한계점을 가진다.

둘째로, 'Merlin Bird ID' 또한 Cornell Lab에서 제공하는 서비스로, 새의 울음소리와 사진으로 동정할 수 있는 것이 특징인 서비스이다. 새 사진뿐만 아니라 울음소리로도 동정할 수 있다는 점에서 동정 성공 가능성을 크게 높인다는 강점을 가지고 있다. 하지만 두 가지 정보를 동시에 이용해 검색하는 것은 아니기 때문에 동정 정확도는 해당 서비스의 데이터 분석 수준에 의존적이다. 또한 해당 서비스는 사용자의 탐조 기록을 관리하는 기능이 없어 사용자는 탐조 기록을 관리하기 위해 여전히 직접 분류하고 저장해야 한다는 한계점을 가진다.

마지막으로 세 서비스 중 가장 최근인 2025년 11월에 출시된 '새록'은 우리나라에서 개발된 서비스로, 사용자가 탐조 일지를 기록하고 관리하는 데에 중점을 맞춰 앞서 소개한 서비스들에 비해 관리형 서비스를 제공하는 것이 특징이다. 해당 서비스 어플리케이션을 다운받아 사용해본 결과, 이 서비스에는 새 사진이나 새 소리를 통해 동정하는 기능은 없었고 사용자가 탐조 일지를 등록할 때 새 사진과 함께 직접 해당 기록의 새 종을 입력하는 기능이 있었다. 이 서비스는 탐조 기록을 어플에 등록하여 탐조인이 직접 새 종을 분류하고 사진 파일을 수기로 정리해야 한다는 문제를 해결했지만, 사용자가 직접 새 종을 입력해야 한다는 점, 따라서 탐조 입문가의 동정 난이도를 낮추지 못한다는 점에서 한계점을 가진다.

기존 서비스와의 차이점

	TEAM 10 Birder	기존 서비스		
				
실시간 종 판별	✓		✓	
관측 기록 관리	✓			✓
실시간 정보 공유	✓			✓
종별/지역별 검색	✓	✓	✓	

II. Birder-Summary

3. Birder에서 제공하고자 하는 서비스

1) AI를 이용한 실시간 종 식별 서비스

Birder는 사용자에게서 사용자가 동정하고자 하는 새 사진을 받아, AI를 통해 새 종을 식별해 해당 새 종에 대한 정보를 제공하고자 한다. AI에 사용자에게서 받은 원본 사진을 제공하여 정확한 판별 요청을 보내고, 신뢰도에 따른 5개의 후보를 요청한다. AI로부터 받은 후보들을 사용자에게 제공해 사용자로부터 직접 관측한 새의 종과 일치하는 후보를 입력받는다.

2) 개인별 관측 도감 서비스

Birder는 사용자에게 개인별 관측 기록을 관리할 수 있는 도감 서비스를 제공하고자 한다. 대한민국에서 관측되는 새 종 리스트를 확보한 뒤, 사용자가 탐조에 성공한 새들만을 리스트에 업데이트하여, 사용자가 직접 등록한 사진들을 이용해 리스트를 채워 넣어 도감을 제공한다. 또한 사용자가 등록한 사진에서 위치 정보와 시간 정보를 추출해 자동으로 저장해주어 편의성을 극대화한다.

3) 실시간 관측 기록 DB 구축 및 제공 서비스

Birder는 사용자로부터 실시간 관측 기록 데이터를 받아 이를 통합적으로 가공하여 대한민국 관측 기록 DB 서비스로 제공하고자 한다. Birder는 사용자가 Birder를 통해 새 사진 검색을 성공했을 경우, 해당 새 사진 관측 기록의 위치 정보와 관측 날짜 정보를 DB에 저장한다. 이는 사용자의 관측 도감에 업데이트될 뿐 아니라 Birder의 전체 관측 DB에 저장되어 실시간 관측 정보를 업데이트한다. 이러한 사용자 개개인의 관측 기록이 모여 대한민국 전체 탐조 DB가 구축되고, 사용자는 자신이 직접 탐조한 기록 외에도 다른 사용자의 탐조 기록을 검색할 수 있다. 이 서비스는 이러한 과정으로 구축된 DB를 사용자의 개인정보 보호를 위해 관측 기록의 위치 정보와 날짜 정보만 가공하여 제공하는 것이다.

II. Birder-Summary

4. 기대 효과 및 의의

1) AI를 이용한 실시간 종 식별 서비스의 기대효과 및 의의

AI를 통한 실시간 동정 서비스는 사용자로 하여금 탐조의 접근성을 크게 높인다. 백과사전을 뒤적이다 탐조하려는 새의 자취를 놓칠 가능성을 크게 줄여 리스크 없는 탐조를 가능하게 한다. 이는 곧 관측 기회 손실을 방지하여 탐조의 입문 장벽을 크게 낮춘다는 의의를 갖는다. 또한 신속한 새 종 판별을 통해 사용자의 종 식별 숙련도를 키울 수 있을 것으로 기대한다.

2) 개인별 관측 도감의 기대효과 및 의의

개인별 관측 도감은 Birder를 통해 동정만 하여도 기본으로 축적되는 개인 탐조 DB이다. 동정에 성공했을 때 생성되는 관측 위치와 시간이 기록된 로그는 도감을 채워나가는 재미라는 동기를 부여한다. 이러한 재미는 탐조라는 취미에 크게 의존적이지 않은 재미를 생산한다는 점에서 새로운 탐조 입문자와 사용자 풀 확대를 기대해 볼 수 있다. 또한 이 서비스는 탐조 입문자만 겪는 문제가 아닌 모든 탐조인이 겪는 기록의 번거로움과 기록 누락의 가능성을 제거한다는 점에서 큰 의의를 갖는다.

3) 실시간 관측 기록 DB의 기대효과 및 의의

실시간 관측 기록 DB는 현재로서 빈약한 새 관측 DB를 모든 사용자의 실제 관측 기록을 모아 구축하여 탐조 계획을 위한 유의미한 정보를 제공한다는 의의를 갖는다. 실제 사용자에게서 계속해서 새로운 관측 기록을 받는다는 점에서 데이터의 최신성을 확보하고, 개인정보를 제외한 최대한 자세한 정보를 제공한다는 점에서 개인별로 목적에 맞게 검색하여 DB를 활용할 수 있다. 이러한 자세한 DB는 탐조인으로 하여금 탐조 난이도를 낮춰 탐조의 활성화를 기대해 볼 수 있다.

II. Birder-Summary

5. 주요 기능 리스트

1) AI를 이용한 실시간 종 식별

Flutter를 통해 사용자에게서 사용자가 검색하려는 새 사진을 촬영받거나 업로드받아 원본 파일을 django RESTframework BE로 전송한다. django에서 해당 사진을 supabase storage에 저장하여 public url을 받아오며, 원본 파일을 OpenAI 모델로 전송해 해당 종 식별을 요청한다. 이때 django는 단순히 이미지를 전달하는 역할만 하는 것이 아니라, 이미지의 EXIF 정보를 확인하여 촬영 시간과 위치 정보가 존재하는 경우 이를 함께 추출한다. 추출된 메타데이터는 이후 관측 기록 저장과 지역 기반 검색에 활용된다. OpenAI 모델에 Birder가 보유하고 있는 새 525종 내로 결과를 제한시켜 정확도순으로 최대 5개의 후보를 요청하고, django에서 반환된 후보종들을 DB에 저장된 새 종들의 정보와 매칭시켜 가공한다. 가공된 결과값은 다시 Flutter를 통해 사용자에게 제공되고, 사용자는 반환된 5개의 후보들을 스와이프하여 해당 후보들 중 사용자가 관측한 새 종과 일치하는 정보가 있는지 확인한다. 5개의 후보 중 일치하는 새 정보가 있다면 해당 후보에서 '이 새가 맞아요' 버튼을 이용해 서버로 결과값을 전송하고, 이어 관측 도감을 업데이트한다. 만약 일치하는 새 정보가 없다면 '이 중에 없어요' 버튼을 이용해 서버로 결과값을 전송하며 DB 저장공간 효율을 위해 django는 이미 supabase storage에 업로드된 원본 파일을 삭제한다.

2) 개인별 관측 도감

Birder는 사용자에게 개인별 관측 기록을 관리할 수 있는 도감 서비스를 제공하고자 한다. 대한민국에서 관측되는 새 종 리스트를 확보한 뒤, 사용자가 탐조에 성공한 새들만을 리스트에 업데이트하여, 사용자가 직접 등록한 사진들을 이용해 리스트를 채워 넣어 도감을 제공한다. 사용자에게서 새 동정 정보를 받은 django는 사용자가 일치한다고 답변한 새 정보와 사용자가 식별 요청한 새 사진의 public url을 이용해 supabase DB의 log 테이블을 업데이트한다. 사용자는 메인 화면에서 'My Log' 버튼을 이용해 도감 페이지에 접속할 수 있고, 해당 페이지에서 현재까지 사용자가 관측한 새 목록을 확인할 수 있다. 사용자가 해당 페이지에 접속할 때 Flutter로부터 요청을 받아 django는 supabase의 log 테이블에서 사용자를 기준으로 필터링하여 로그인한 사용자가 관측한 로그들을 모두 반환한다. Flutter는 반환받은 로그 중 아직 사용자가 관측하지 못한 종은 기본 화면에 새 종만 출력하고, 사용자가 관측한 새 종은 사용자가 가장 최근에 관측 성공한 사진으로 출력한다. 이를 통해 사용자는 자신이 이미 발견한 종과 아직 발견하지 못한 종을 한눈에 구분할 수 있으며, 관측한 종이 늘어날수록 개인 도감이 채워지는 구조를 가진다. 사용자가 특정 종 도감을 상세 조회 요청을 하면 django는 해당 요청에 따라 log 테이블에서 사용자와 요청된 종을 기준으로 필터링하여 관측 기록을 반환하고, 이를 이용해 Flutter는 해당 사용자가 역대 관측한 모든 로그의 관측 위치와 관측 날짜와 관측별 사용자가 검색한 사진을 출력한다. 이 외에도 대한민국 지도를 통해 직관적으로 사용자의 지역별 관측 밀도와 관측한 사진을 한 눈에 파악할 수도 있다.

3) 실시간 관측 기록 DB

Birder는 사용자로부터 실시간 관측 기록 데이터를 받아 이를 통합적으로 가공하여 대한민국 관측 기록 DB 서비스로 제공하고자 한다. Birder는 사용자가 Birder를 통해 새 사진 검색을 성공할 경우, 해당 새 사진 관측 기록의 위치 정보와 관측 날짜 정보를 supabase DB에 저장한다. 이는 사용자의 관측 도감에 업데이트될 뿐 아니라 Birder의 전체 관측 DB에 저장되어 실시간 관측 정보를 업데이트한다. 즉, 개인 사용자의 관측 기록은 개인 도감의 데이터가 되는 동시에, 전체 사용자 기반 관측 DB를 구성하는 데이터가 된다. 이러한 사용자 개인의 관측 기록이 모여 대한민국 전체 탐조 DB가 구축되고, 사용자는 자신이 직접 탐조한 기록 외에도 다른 사용자의 탐조 기록을 검색할 수 있다. 이 서비스는 이러한 과정으로 구축된 DB를 사용자의 개인정보 보호를 위해 관측 기록의 위치 정보와 날짜 정보만 가공하여 제공하는 것이다.

우선 사용자가 종별 검색을 시도한다면 Flutter는 해당 사용자가 검색하려는 특정 종을 받아 django로 요청한다. 해당 요청을 받은 django는 supabase DB의 log 테이블을 종별로 필터링하여 log별 사용자의 정보는 제거된 결과값을 요청한다. 반환된 로그의 관측 위치와 관측 날짜와 로그별 supabase storage에 저장된 사진의 public url을 Flutter에 응답으로 전송한다. 이를 통해 사용자는 반환된 결과 로그들을 개별로 조회하여 로그별 실제 관측 사진을 확인할 수 있다.

이와 비슷하게 사용자가 위치별 검색을 시도한다면 Flutter는 해당 사용자가 검색하려는 특정 위치를 받아 django로 요청한다. 해당 요청을 받은 django는 supabase DB의 log 테이블에서 photo 테이블을 참조하여 위치별로 필터링한 결과값을 요청한다. 사용자는 반환받은 결과들을 이용해 특정 행정구역에서 어떤 새 종이 많이 관측되었는지 확인할 수 있다.

결과적으로 실시간 관측 기록 DB 기능은 사용자가 등록한 개별 관측 로그를 누적하여, 종별 검색, 지역별 검색, 기역별 필터링, 지도 기반 조회가 가능한 공용 관측 DB로 확장하는 기능이다. 이를 통해 Birder는 단순한 개인 도감 앱을 넘어, 사용자 참여형 조류 관측 데이터 플랫폼으로 기능할 수 있다.

III. Project-Design

1. 요구사항 정의서

<https://docs.google.com/spreadsheets/d/1lQ3OICnJPZoBKrn-PLKXX7kWE6cKakk-sadHyT1r2cE/edit?usp=sharing>

JL Birder 요구사항 정의서					
어플리케이션명	Birder			작성자	최이준, 최유선, 이운성
대분류	중분류	소분류	요구사항명	요구사항내용	세부내용
로그인	로그인 입력		ID 입력 기능	사용자가 ID 입력할 수 있는 입력 필드 제공	텍스트 입력, 글자 수 제한 설정, 영문&숫자만 허용, 클릭시 커서 깜빡임 표시
			PW 입력 기능	사용자가 PW 입력할 수 있는 입력 필드 제공	입력값 마스크, 영문&숫자만 허용, 클릭시 커서 깜빡임 표시
	로그인 동작	로그인 실패 처리	로그인 요청	사용자가 ID/PW 입력 후 "LOGIN" 버튼 누르면 서버에 인증 요청	ID/PW 둘 중 하나라도 입력하지 않으면 경고창 출력, 로그인 API 호출, 성공 시 메인 화면 이동
			ID, PW, PW 확인, 이메일 입력	ID, PW, PW 확인, 이메일 입력	ID/PW 둘 중 하나라도 불일치 시 경고창 출력
계정 찾기	계정 찾기		ID 찾기 진입	"ID찾기" 버튼 클릭 시 ID/PW 찾기 페이지로 이동	ID/PW 찾기 화면 호출, 뒤로가기 시 로그인 화면 복귀
			PW 찾기 진입	"PW찾기" 버튼 클릭 시 ID/PW 찾기 페이지로 이동	ID/PW 찾기 화면 호출, 뒤로가기 시 로그인 화면 복귀
	회원 구성	로그인 화면 UI	로그인 화면의 기본 UI	로그인 화면의 기본 UI	Birder 로고 표시, ID & PW 입력 필드, LOGIN 버튼, ID찾기버튼 / PW찾기버튼
			이메일 입력 기능	사용자가 이메일 입력할 수 있는 입력 필드 제공	텍스트 입력, 한글&영문만 허용
ID/PW 찾기 화면	정보 입력	ID, PW	ID, 이메일 입력 기능	사용자가 ID, 이메일 입력할 수 있는 입력 필드 제공	영문&숫자 허용
			ID 입력 기능	회원가입 시 ID 입력할 수 있는 필드 제공	텍스트 입력, 영문&숫자만 허용, 5~16글자 수 제한
	회원가입 입력		ID 중복 확인	사용자가 입력한 ID의 사용 가능 여부 확인	"중복확인" 버튼 클릭 시 중복 체크 API 호출, 사용 가능/불가능 여부 팝업창 출력
			PW 입력 기능	회원가입 시 PW 입력할 수 있는 필드 제공	입력값 마스크, 영문&숫자만 허용
회원가입	회원가입		PW 확인 입력 기능	사용자가 입력한 PW와 동일한 PW인지 확인	PW와 동일한지 비교, 불일치 시 회원가입 버튼 클릭시 경고창 출력
			휴대전화번호 입력 가능	사용자가 휴대전화번호 입력할 수 있는 필드 제공	숫자만 허용, 자라수 검증
	본인 확인		휴대전화 인증	인증번호를 통한 휴대전화 인증 가능	"인증" 버튼 클릭 시 인증번호 발송 API 호출, 인증번호 입력 & 확인창 출력
			회원가입 요청	회원가입 요청을 서버로 전달	회원가입 API 호출, 성공 시 회원가입 성공창 출력
회원 구성	회원가입 UI		회원가입 화면의 기본 UI	회원가입 화면의 기본 UI	타이틀 "회원가입" 표시, ID/PW이름(전화번호)/생년월일 필드 표시, "회원가입" 버튼 표시
			촬영	앱 내 촬영 화면으로 전환, 다중 촬영 불가, 1회 촬영 시 해당 사진으로 만별을 시도할 것인지를 확인	
	새 검색		사진 업로드	가까이 이미지 파일에 접근, 이미지 클릭 시 해당 이미지의 이미지에 담긴 위치 정보 전송	업로드한 사진에서 위치정보가 없을 경우, 사용자에게서 위치정보를 받을지 확인
			취소 버튼	클릭 시 새 검색 버튼 클릭 이전으로 전환	
마이 로그	MY Log		도감화면 이동 버튼	도감 화면으로 화면 이동	
			Birders Log	새 로그 검색 및 화면으로 이동	
	설정		설정 화면 이동 버튼	설정 화면으로 이동	
			발견 새 목록	도감 내 중첩 목록 분류하여 목록 제공	발견한 중첩 사진으로, 발견하지 못한 중첩 목록 표시
My log	개인 도감		새 검색	도감 내 중첩 검색 가능	
			지도 버튼	지구 아이콘 버튼 클릭 시 지도 도감 화면으로 이동	
	발견 로그 상세		발견 위치 표시	업로드 된 사진의 위치 정보로 위도/경도와 지역명 표시	
			발견 시간 표시	업로드 된 사진의 촬영 날짜, 시간 표시	
Birders Log	중첩 정보 상세		중첩 상세 정보 표시	중첩 이름(국명/학명), 기본 특징, 서식지 등 간략한 정보를 합성으로 제공	
			최근 로그 표시	사용자의 업로드 로그 가장 최근의 간략 일지와 위치를 표시	
	지도 도감		지도 기반 관찰 기록	사용자가 촬영한 새 이미지를 위도/경도에 따라 지도 위에 표시	이미지 클릭 시 탐조 로그 상세 화면으로 이동
			지역별/종별 검색 버튼	지역별로 검색할 지 종별로 검색할 지 선택할 수 있는 버튼 두 개	
지역별 보기	대한민국 지도 화면		지역별 검색	키워드 검색이 아닌 클릭 상호작용을 통한 검색	
			지역명 검색	(도/시)로 구분되어 있는 대한민국 지도 화면 출력	
	특정 지역		지도 화면	선택된 지역에서 실제 관측된 새 목록을 중첩으로 누적 관측 필수 함께 출력	
			전체 로그	해당 종이 해당 지역에서 관측된 결과를 최신순으로 정렬해 출력, 관측 일지와 관측 위치를 경도 위도 혹은 지역명을 간단하게 출력(구 단위까지)	
종별 보기	종/지역/기간 필터링 된 결과 로그 화면		특정 로그	전체 로그 화면에서 특정 로그 클릭 시 해당 로그를 추가하는데 사용된 사용자의 새 관측 사진과 새 중첩 합성으로 출력	
			종별 검색	새 종을 텍스트로 검색 가능	사용자가 새 종 검색에 선택해 용이하도록 인기검색어, 오늘날까지/계절 기준 검색어, 최근 가장 많이 추가된 새 검색어 추천
	지도 그래프 화면		지도 그래프 화면	새 종을 클릭해 지도를 필터링으로 확인 가능, 지도(구)으로 구별되어 있는 지도를 클릭해 지역별 관측 기록 확인	지도 상단에 검색할 관측 기간 선택 가능
			전체 로그	해당 종이 해당 지역에서 관측된 결과를 최신순으로 정렬해 출력, 관측 일지와 관측 위치를 경도 위도 혹은 지역명을 간단하게 출력(구 단위까지)	
상단바	로그		로그	전체 로그 화면에서 특정 로그 클릭 시 해당 로그를 추가하는데 사용된 사용자의 새 관측 사진과 새 중첩 합성으로 출력	만약 일반 사용자가 해당 로그가 잘못된 것을 발견하면 신고 버튼을 통해 해당 종 기록이 잘못됐음을 알릴 수 있음
			로그인	로그인이 완료된 어플 런딩페이지로 귀환	
	설정 버튼		설정 화면으로 이동	설정 화면으로 이동	
			도감 버튼	마이 Log 화면 이동 버튼	
하단바	마이 Log 버튼		마이 Log 버튼	마이 Log 화면 이동 버튼	
			마이 Log 버튼	마이 Log 화면 이동 버튼	
	마이 Log 버튼		마이 Log 버튼	마이 Log 화면 이동 버튼	
			마이 Log 버튼	마이 Log 화면 이동 버튼	
설정 화면	알림 설정		알림 설정	사용자가 앱 내 알림 ON/OFF 가능하게 설정	스위치 토글, 토글 정보 바로 반영, 알림 권한 필요 시 안내 팝업 호출
			위치 서비스	위치 정보 사용 여부 결정	토글 ON/OFF 제공, 최초 ON 시 OS 위치권한 요청
	로그인 정보 이동		로그인 정보 이동	"로그인 정보" 메뉴 선택 시 로그인 정보 화면으로 이동	화면 전환, 뒤로가기 시 설정 화면 복귀
			로그아웃	로그아웃 클릭 시 로그아웃 팝업 창 표시 후 로그아웃 처리	"로그아웃 되었습니다" 팝업창 표시, 확인 클릭 시 로그인 화면으로 이동
로그인 정보 화면	로그인 정보 표시		로그아웃	로그아웃 클릭 시 로그아웃 팝업 창 표시 후 로그아웃 처리	ID, 텍스트 표시, PW 변경 : 버튼 표시, 비밀번호 : 버튼 표시
			로그아웃	로그아웃 클릭 시 로그아웃 팝업 창 표시 후 로그아웃 처리	ID, 텍스트 표시, PW 변경 : 버튼 표시, 비밀번호 : 버튼 표시
	PW 변경 이동		로그아웃	로그아웃 클릭 시 로그아웃 팝업 창 표시 후 로그아웃 처리	ID, 텍스트 표시, PW 변경 : 버튼 표시, 비밀번호 : 버튼 표시
			로그아웃	로그아웃 클릭 시 로그아웃 팝업 창 표시 후 로그아웃 처리	ID, 텍스트 표시, PW 변경 : 버튼 표시, 비밀번호 : 버튼 표시
비밀번호 변경	비밀번호 변경		비밀번호 변경	비밀번호 변경 화면 이동	화면 전환, 뒤로가기 시 로그인 정보 화면 복귀
			비밀번호 변경	비밀번호 변경 화면 이동	화면 전환, 뒤로가기 시 로그인 정보 화면 복귀
	회원가입 이동		회원가입 이동	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공
			회원가입 이동	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공
회원가입	회원가입		회원가입	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공
			회원가입	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공
	회원가입		회원가입	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공
			회원가입	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공
회원가입	회원가입		회원가입	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공
			회원가입	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공
	회원가입		회원가입	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공
			회원가입	회원가입 이동 화면 표시	팝업 창 안에 내/나/나/나 버튼 제공

요구사항 정의서는 사용자가 Birder에서 수행하는 주요 활동을 기준으로 My Log와 Birder's Log 영역으로 구분하여 작성하였다. My Log는 개인 사용자의 관측 경험을 관리하는 영역이며, Birder's Log는 여러 사용자의 관측 기록을 바탕으로 지역별/종별 탐조 정보를 탐색하는 영역이다.

My Log 영역은 사용자가 직접 발견한 조류를 개인 도감 형태로 확인할 수 있도록 구성하였다. 사용자는 발견한 새 목록을 통해 자신이 관측한 종과 아직 관측하지 못한 종을 구분할 수 있으며, 특정 종을 선택하면 해당 종에 대한 관측 사진, 위치, 시간, 최근 로그를 확인할 수 있다. 또한 동일한 종을 여러 번 관측한 경우에는 각 관측 기록을 개별 로그로 관리하여 사용자의 탐조 이력을 누적할 수 있도록 하였다. 이를 통해 My Log는 단순한 사진 저장 기능이 아니라, 사용자의 개인별 탐조 기록을 체계적으로 정리하는 기능을 담당한다.

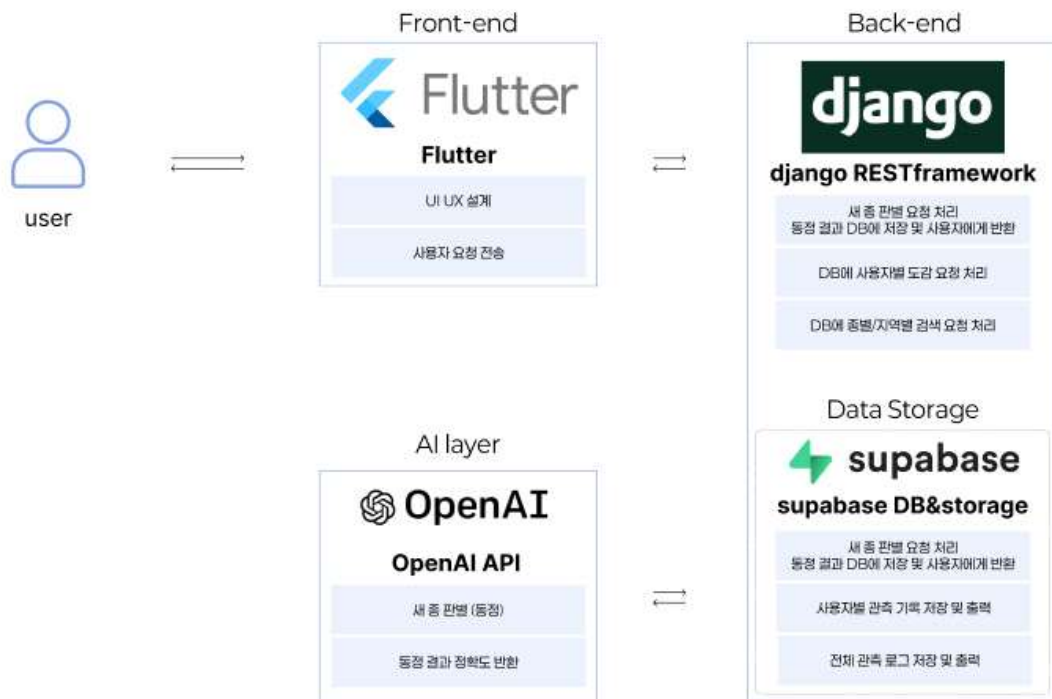
Birder's Log 영역은 전체 사용자의 관측 기록을 바탕으로 조류 데이터를 탐색할 수 있도록 설계하였다. 사용자는 지역별 보기 기능을 통해 대한민국 지도에서 특정 지역을 선택하거나 지역명을 검색하여 해당 지역의 관측 기록을 확인할 수 있다. 또한 종별 보기 기능을 통해 특정 조류 종을 검색하고, 해당 종이 어느 지역에서 얼마나 관측되었는지 지도와 로그 목록을 통해 확인할 수 있다. 지역, 종, 기간 조건을 함께 적용할 수 있도록 하여 사용자가 원하는 조건의 관측 데이터를 보다 구체적으로 조회할 수 있게 하였다.

결과적으로 요구사항 정의서는 사용자의 관측 기록을 개인 단위로 관리하는 기능과, 누적된 관측 데이터를 전체 사용자 기반으로 탐색하는 기능을 함께 반영한다. 이를 통해 Birder는 개인 탐조 기록 도구이면서 지역별/종별 조류 관측 정보를 공유하고 조회할 수 있는 관측 데이터 플랫폼으로 동작하도록 설계되었다.

III. Project-Design

2. Birder의 시스템 구성도

1) 전체 SW 구조 설명



SW 구조는 크게 Front-end, Back-end, AI Layer, Data Storage로 구성된다. 사용자는 Flutter 기반의 애플리케이션을 통해 사진 업로드, 조류 종 후보 확인, 개인 도감 조회, 관측 기록 검색 등의 기능을 사용한다. Flutter는 사용자 인터페이스를 제공하고, 사용자의 요청을 Django REST Framework 기반 Back-end 서버로 전달하는 역할을 한다.

Back-end는 전체 서비스의 제어 흐름을 담당한다. Django REST Framework는 Front-end로부터 전달된 요청을 처리하고, 필요한 경우 OpenAI API 또는 Supabase와 통신한다. 예를 들어 사용자가 조류 사진을 업로드하면 Django 서버는 이미지를 수신한 뒤, 사진 메타데이터를 추출하고, 이미지를 Supabase Storage에 저장하며, AI 식별 요청을 OpenAI API로 전달한다. 이후 OpenAI API로부터 받은 후보 종 결과를 정제하여 다시 Front-end로 반환한다.

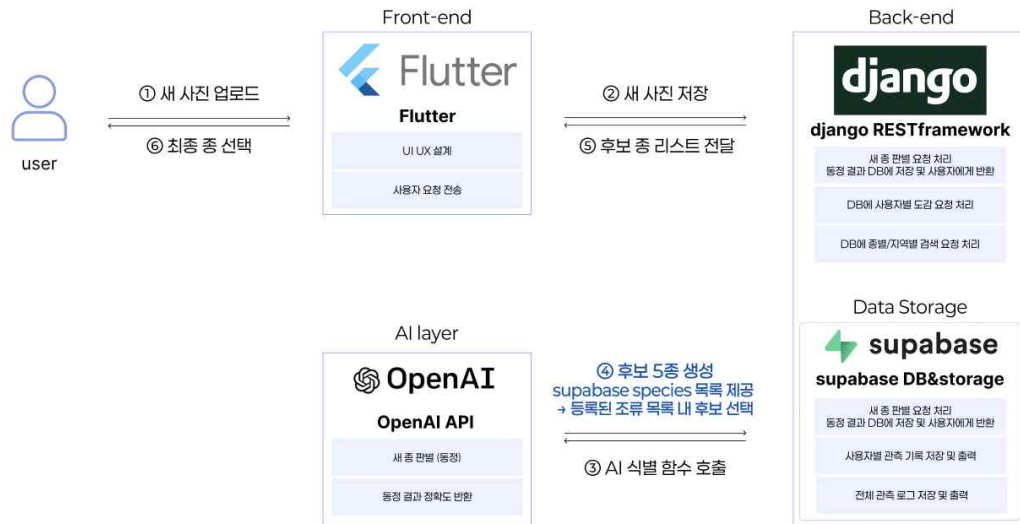
AI Layer는 OpenAI API를 이용한 조류 종 식별 기능을 담당한다. 사용자가 업로드한 조류 이미지는 Django 서버를 거쳐 OpenAI API에 전달되며, OpenAI API는 사진 속 조류와 유사한 후보 종을 반환한다. 이때 후보 종은 텍스트 응답이 아니라, rank, species_code, common_name_ko, scientific_name, confidence를 포함하는 데이터로 처리된다.

Data Storage는 Supabase DB와 Supabase Storage로 구성된다. Supabase Storage는 업로드된 사진 파일을 저장하고, Supabase DB는 사용자 정보, 조류 종 정보, 사진 메타데이터, 관측 로그를 저장한다. 사진 업로드 시 서버는 이미지를 Storage에 저장한 뒤, 이미지 URL과 위도, 경도 등의 정보를 photo 테이블에 저장한다. 이후 사용자가 AI 후보 중 최종 종을 선택하면 photo_num, species_code, reg_date, 사용자 ID가 log 테이블에 저장된다.

2) 주요 기능별 제어/데이터 흐름

2.1) AI 이용 실시간 종 식별 기능

[주요 기능 1] AI 종 식별



AI 기반 실시간 종 식별 기능은 사용자가 조류 사진을 업로드하면 서버가 사진을 저장하고, OpenAI API를 통해 후보 종을 분석한 뒤, 그 결과를 Front-end에 반환하는 흐름으로 동작한다.

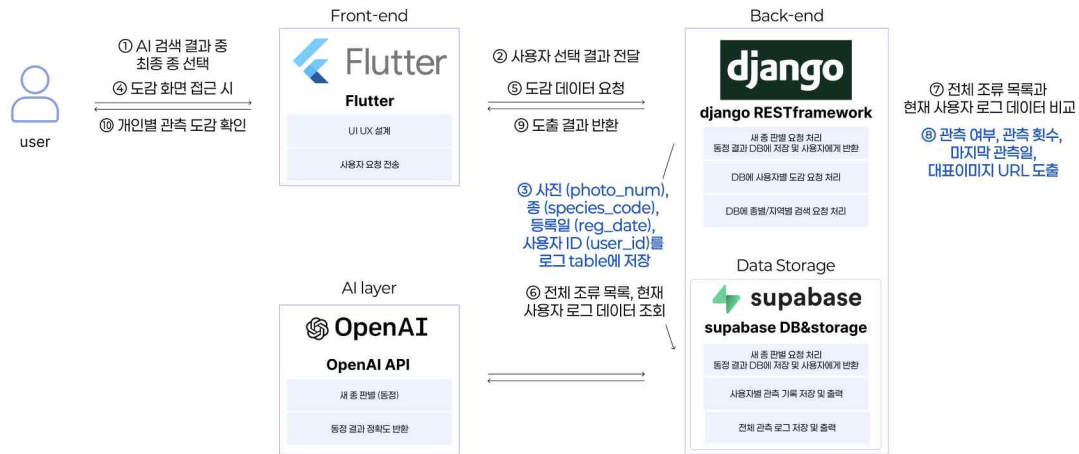
먼저, 사용자는 조류 사진을 선택하거나 촬영하여 업로드한다. Flutter는 해당 이미지 파일을 multipart/form-data 형태로 Django Back-end의 사진 업로드 API에 전송한다. Django 서버는 이미지를 수신한 뒤, EXIF 데이터에서 위도, 경도, 촬영 시간을 추출한다. 이후 이미지를 Supabase Storage에 업로드하고, 업로드된 이미지의 public URL과 위치, 시간 정보를 Supabase의 photo 테이블에 저장한다.

사진 저장이 완료되면 Django 서버는 동일한 이미지 파일을 AI 식별 함수에 전달한다. AI 식별 함수는 이미지를 base64 형식으로 변환하여 OpenAI API 요청에 포함하고, Supabase의 species 목록을 함께 전달하여 AI가 등록된 조류 목록 안에서 후보 종을 선택하도록 제안한다. OpenAI API는 사진 속 조류와 유사한 후보 종 5개를 반환하며, 서버는 반환된 후보를 species_code 기준으로 검증하고 최종 후보 리스트를 구성한다.

이후 서버는 각 후보 종의 species_code를 기준으로 Supabase의 species 테이블에서 상세 설명을 추가 조회한다. 최종적으로 Django 서버는 후보 종 리스트와 사진 번호인 photo_num을 Flutter에 반환한다.

2.2) 개인별 관측 도감 기능

[주요 기능 2] 개인별 관측 도감



개인별 관측 도감 기능은 사용자가 AI 후보 중 최종 종을 선택하면 해당 결과를 사용자별 관측 로그를 저장하고, 이후 로그인한 사용자의 관측 이력을 바탕으로 개인 도감을 제공하는 흐름으로 동작한다.

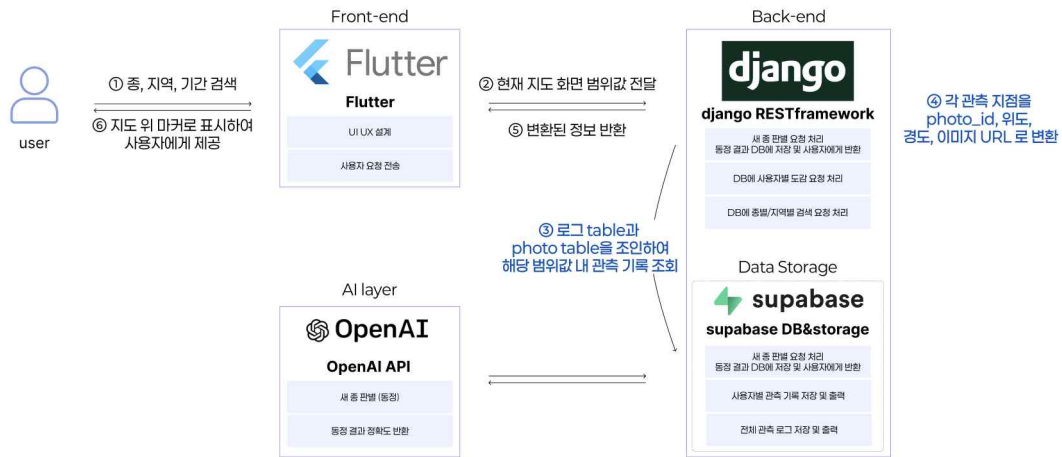
사용자가 Flutter 화면에서 AI 후보 중 하나를 선택하면, Flutter는 선택된 species_code와 해당 사진의 photo_num을 Django 서버로 전송한다. Django 서버는 전달 받은 정보를 Supabase의 log 테이블에 저장한다. 이 과정을 통해 하나의 관측 기록은 “사용자-사진-종-등록 시간” 형태로 연결된다.

이후 사용자가 개인 도감 화면에 접근하면 Flutter는 도감 데이터를 Django 서버에 요청한다. Django 서버는 Supabase의 species 테이블에서 전체 조류 목록을 조회하고, 동시에 현재 로그인한 사용자의 log 데이터를 조회한다. 서버는 species 목록과 사용자 log 데이터를 비교하여 각 종에 대해 관측 여부, 관측 횟수, 마지막 관측일, 대표 이미지 URL을 도출한다. 도출된 결과는 order_name 기준으로 그룹화되어 Flutter에 반환된다.

사용자가 도감에서 특정 종을 선택하면 Django 서버에서는 해당 species_code에 대한 종 기본 정보를 조회하고, 현재 사용자의 log 테이블에서 해당 종의 관측 기록과 연결된 사진 정보를 함께 가져온다. 이를 통해 Flutter는 특정 종에 대한 개인별 관측 상세 화면을 구성할 수 있다.

2.3) 실시간 관측 기록 DB 제공 기능

[주요 기능 3] 실시간 관측 기록 종별/지역별 검색



실시간 관측 기록 DB 제공 기능은 여러 사용자가 등록한 관측 기록을 Supabase DB에 누적하고, 이를 종별, 지역별, 지도 범위별로 조회할 수 있도록 하는 흐름으로 동작한다.

관측 기록의 생성은 AI 식별 결과를 사용자가 확정하는 시점에서 시작된다. 사용자가 후보 종 중 하나를 선택하면 Django 서버는 선택된 종의 species_code, photo_num, 사용자 ID, 등록 시간을 log 테이블에 저장한다.

지도 기반 관측 기록 조회는 Flutter가 현재 지도 화면의 범위 값을 Django 서버에 전달하는 방식으로 이루어진다. Django 서버의 MapPointsViews는 전달받은 bbox 좌표값이 올바른지 검증한 뒤, Supabase의 log 테이블과 photo 테이블을 조인하여 해당 지도 범위 안에 있는 관측 기록만 조회한다. 서버는 각 관측 지점을 photo_id, 위도, 경도, 이미지 URL 형태로 변환하여 Flutter에 반환하고, Flutter는 이를 지도 위의 마커로 표시한다.

종별 관측 기록 조회는 사용자가 특정 종과 기간을 선택하면 Django 서버가 해당 조건에 맞는 관측 로그를 조회하는 방식으로 동작한다. 지역별 관측 기록 조회는 사용자가 지역명, 종 코드, 기간 조건을 전달하면 Supabase에서 해당 조건에 맞는 누적 관측 통계 또는 상세 로그를 반환하는 방식으로 동작한다.

III. Project-Design

3. 주요 엔진 및 기능 설계

1) ChatGPT 기반 조류 식별 모듈 구현 방법

ChatGPT 기반 조류 식별 모듈은 사용자가 업로드한 조류 사진을 분석하여 후보 조류 종을 반환하는 모듈이다. 이 모듈은 본 프로젝트의 핵심 기능 중 AI를 이용한 실시간 종 식별 기능을 담당한다.

해당 모듈은 Django Back-end에서 OpenAI API를 호출하고, AI 응답을 프론트엔드에서 사용할 수 있는 후보 종 데이터로 정제하는 방식으로 구현하였다.

먼저 사용자가 조류 사진을 업로드하면 UploadBirdPhotoView에서 이미지를 수신한다. 서버는 이미지 파일에서 EXIF 데이터를 추출하여 위도, 경도, 촬영 시간을 가져오고, 이후 이미지를 Supabase Storage에 업로드한다. 업로드된 이미지의 public URL과 위치/치간 정보는 Supabase의 photo 테이블에 저장된다.

사진 저장이 완료되면 서버는 동일한 이미지 파일을 identify_bird(image) 함수에 전달하여 AI 식별을 수행한다. AI 식별 결과는 후보 종 리스트 형태로 반환되며, 각 후보는 순위, 조코드, 국명, 학명, 신뢰도, 대표 이미지 URL을 포함한다. 이후 각 후보의 species_code를 기준으로 Supabase의 species 테이블에서 상세 설명을 추가 조회하여 후보 정보에 포함한다. 최종적으로 서버는 후보 종 리스트와 photo_num을 함께 프론트엔드에 반환한다.

OpenAI API와 직접 통신하는 식별 함수에서는 먼저 응답 구조를 안정적으로 유지하기 위해 Pydantic 기반의 candidate, IdentifyResult 스키마를 정의하였다. Candidate는 후보 한 종의 정보를 나타내며 rank, species_code, common_name_ko, scientific_name, confidence 값을 포함한다. IdentifyResult는 후보 종 5개를 포함하는 응답 구조로 정의되어, AI 응답이 항상 정해진 JSON 형식으로 파싱되도록 하였다.

또한 Supabase의 species 테이블에서 가져온 종 목록을 species_code, common_name_ko, scientific_name 형식의 문자열로 압축하여 OpenAI 프롬프트에 포함하였다. 이를 통해 AI가 임의의 종을 생성하지 않고, DB에 등록된 종 목록 안에서만 후보를 선택하도록 제한하였다. 응답 이후에도 서버 측에서 species_code를 기준으로 whitelist 필터링을 한 번 더 수행하여 허용되지 않은 종이 최종 결과에 포함되지 않도록 처리하였다.

만약 필터링 후 후보가 5개보다 적을 경우에는 Supabase species 목록에서 중복되지 않는 종을 낮은 confidence 값으로 보충하여 항상 5개의 후보가 반환되도록 구현하였다. 이는 프론트엔드에서 항상 동일한 개수의 후보 카드를 표시할 수 있도록 하기 위한 처리이다. 또한 최종 후보 5개에 대해서는 학명 기반으로 Wikimedia Commons API를 조회하여 대표 이미지 URL을 추가하였다.

2) 행정구역 정규화 모듈

행정구역 정규화 모듈은 사진에서 추출한 위도, 경도 좌표를 행정구역 단위의 문자열로 변환하는 모듈이다. 이 모듈은 본 프로젝트의 핵심 기능 중 실시간 관측 기록 DB 제공 기능과 직접적으로 연결된다.

먼저 사용자가 업로드한 사진에서 EXIF 데이터를 추출한다. EXIF 데이터에 GPS 정보가 존재하는 경우 GPSLatitude, GPSLatitudeRef, GPSLongitude, GPSLongitudeRef 값을 읽고 이를 십진수 형태의 위도, 경도 값으로 변환한다. 이 과정은 `get_decimal_from_dms()` 함수와 `extract_exif_data()` 함수에서 처리된다.

이후 추출된 위도와 경도는 `normalize_area_from_latlon()` 함수에 전달된다. 해당 함수는 Nominatim reverse geocoding API에 lat, lon, zoom, addressdetails, accept-language 값을 포함하여 요청을 보낸다. accept-language는 “ko”로 설정하여 한국어 주소 정보를 우선적으로 받을 수 있도록 하였다.

만약 API 응답에서 행정구역 정보가 명확하게 분리되지 않는 경우에는 display_name 값을 쉼표 기준으로 분리하여 대체 주소 문자열을 생성한다. 이때 국가명은 지역 검색에 불필요하므로 stopword로 제거한다. 외부 API 호출 실패, 네트워크 오류, 주소 변환 실패가 발생할 경우에는 “UNKNOWN”을 반환하도록 예외 처리를 적용하여 전체 업로드 및 관측 기록 저장 과정이 중단되지 않도록 하였다.

III. Project-Design

4. 주요 기능의 구현

1) AI를 이용한 실시간 종 식별 기능과 모듈의 연관성

AI를 이용한 실시간 종 식별 기능은 ChatGPT 기반 조류 식별 모듈과 직접적으로 연결된다. 사용자가 조류 사진을 업로드하면 서버는 이미지를 저장한 뒤 AI 식별 함수를 호출하고, OpenAI API를 통해 후보 종 5개를 생성한다. 이 과정에서 Supabase의 species 목록을 함께 제공하여 AI가 등록된 조류 목록 안에서만 후보를 선택하도록 하며, 최종 결과는 후보 종 리스트로 Front-end에 전달된다.

2) 개인별 관측 도감 기능과의 연관성

개인별 관측 도감 기능은 ChatGPT 기반 조류 식별 모듈의 결과를 기반으로 동작한다. AI가 제시한 후보 중 사용자가 최종 종을 선택하면, 서버는 photo_num, species_code, reg_date, 사용자 ID를 Supabase의 log 테이블에 저장한다. 이때 저장된 species_code는 개인 도감에서 사용자가 어떤 종을 관측했는지 판단하는 기준이 된다.

즉, ChatGPT 기반 조류 식별 모듈은 개인별 관측 도감에 기록될 종 정보를 생성하는 출발점이다. 사용자가 확인한 종 정보가 로그로 저장되면, 이후 도감 화면에서는 해당 로그를 기준으로 관측 여부, 관측 횟수, 최근 관측일 등을 계산할 수 있다.

행정구역 정규화 모듈은 개인별 관측 도감 기능과 간접적으로 연결된다. 개인 도감에서 특정 종의 상세 기록을 확인할 때, 사용자는 해당 종을 언제, 어디에서 관측했는지 확인할 수 있어야 한다. 이때 행정구역 정규화 모듈을 통해 좌표 기반 위치 정보가 지역 단위로 정리되면, 관측 기록의 의미가 더 명확해진다.

3) 실시간 관측 기록 DB 제공 기능과의 연관성

실시간 관측 기록 DB 제공 기능은 두 모듈이 모두 관여하는 기능이다. ChatGPT 기반 조류 식별 모듈은 사용자가 관측한 조류의 종 정보를 생성하고, 행정구역 정규화 모듈은 해당 관측이 발생한 위치 정보를 지역 단위로 정리한다. 따라서 두 모듈은 각각 “무엇을 관측했는가”와 “어디에서 관측했는가”를 담당한다.

사용자가 새 종 최종 선택하면 log 테이블에 종 코드와 사진 번호, 등록 시간이 저장된다. 이 로그 데이터는 이후 종별 검색, 특정 종의 누적 관측 조회, 지역별 관측 통계, 지역 기반 관측 기록 조회에 활용된다. 예를 들어 AreaSummaryView는 지역명을 기준으로 Supabase RPC 함수 get_area_observation_summary를 호출하여 지역별 관측 요약 정보를 제공하고, AreaSpeciesLogsView는 지역, 종, 기간 조건을 기반으로 상세 관측 기록을 조회한다.

또한 지도 기반 관측 데이터 제공은 MapPointsView를 통해 구현된다. Front-end가 현재 지도 화면의 범위인 min_lat, min_lng, max_lat, max_lng 값을 전달하면, 서버는 bbox 값의 유효성을 검증한 뒤 Supabase의 log 테이블과 photo 테이블을 조인하여 해당 범위 안에 있는 관측 지점만 조회한다. 반환 데이터에서는 photo_id, 위도, 경도, 이미지 URL이 포함되며 이를 통해 Front-end에서 지도 위에 관측 마커를 표시할 수 있다.

III. Project-Design

5. 기타

Birder에서는 사용자가 업로드한 조류 사진을 분석하기 위해 ChatGPT 기반 OpenAI API를 활용하였다. ChatGPT API는 이미지 입력을 바탕으로 조류 종 후보를 생성하는 AI 식별 엔진으로 사용되었다.

사용자가 Birder 앱에서 조류 사진을 업로드하면 Django 서버의 UploadBirdPhotoView가 이미지 파일을 수신한다. 서버는 먼저 이미지의 EXIF 데이터를 확인하여 위도, 경도, 촬영 시간 정보를 추출하고, 이미지를 Supabase Storage에 업로드한다. 이후 업로드된 이미지의 URL과 메타데이터를 Supabase의 photo 테이블에 저장한다.

사진 저장이 완료되면 서버는 동일한 이미지 파일을 AI 식별 함수인 identify_bird(image)에 전달한다. 이 함수는 내부에서는 이미지를 ChatGPT API가 처리할 수 있도록 base64 형식으로 변환하고 data:image/jpeg;base64,... 형태의 image URL로 구성한다. 이후 OpenAI API 요청 메시지에 이미지 데이터와 식별 요청 문장을 함께 포함하여 모델에 전달한다.

API 요청 시에는 AI가 임의의 조류 종을 생성하지 않도록 Supabase의 species 테이블에 저장된 조류 목록을 함께 제공한다. 각 종 정보는 species_code, common_name_ko, scientific_name 형식으로 정리되어 프롬프트에 포함되며, 모델은 이 목록 안에서만 후보 종을 선택하도록 제한된다. 이를 통해 서비스 DB에 존재하지 않는 종이 결과로 반환되는 문제를 줄일 수 있다.

ChatGPT API의 응답은 Pydantic으로 정의한 구조화된 JSON 형식으로 받도록 구현하였다. 응답 스키마는 Candidate와 IdentifyResult로 구성되며, 각 후보 종은 rank, species_code, common_name_ko, scientific_name, confidence 값을 포함한다. 또한 IdentifyResult는 후보가 항상 5개가 되도록 정의되어있어, Front-end에서 일정한 형식의 결과를 받을 수 있다.

API가 응답을 받은 뒤에는 서버에서 후보 목록을 다시 검증한다. 먼저 rank 기준으로 정렬하고, species_code가 Supabase의 허용 목록에 포함되는지 확인한다. 만약 허용되지 않은 종 코드가 포함되어 있으면 제거하고, 필터링 결과가 5개보다 적으면 DB에 등록된 종 중 중복되지 않는 항목을 낮은 confidence 값으로 보충하여 최종 후보 5개를 보장한다.

마지막으로 최종 후보 5개에 대해 학명을 기준으로 Wikimedia Commons에서 대표 이미지를 조회하고, 각 후보에 wikimedia_image_url 값을 추가한다. 이후 서버는 후보 종 리스트와 사진 번호인 photo_num을 Front-end에 반환된다. 사용자는 이 후보 중 하나를 선택하고, 선택된 species_code는 이후 관측 로그로 저장된다.